

COMMON COURSE ASSIGNMENT

CSA08 – GENERATIVE AI & PROMPT ENGINEERING | Covering Large Language Models & Prompt-Driven Engineering Assistance

A. Assignment Information

Department	Computer Science and Engineering
Programme	B.E. / B.Tech – CSE
Course Code & Course Name	CSA08 – Generative AI and Prompt Engineering
Academic Year / Batch	2026 – 2027
Faculty Name	Dr.Chandravathi C
Assignment Title	Design and Implementation of PromptCraft – A Prompt-Driven Engineering Assistant using Large Language Models and Prompt Engineering
Date of Issue	21-08-2026
Date of Submission	03-09-2026
Maximum Marks	100
Course Outcome(s) – CO	CO1 & CO2
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyze, L5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Prompt-driven AI assistants are increasingly used across the software industry to accelerate coding, debugging, documentation, and requirement analysis, making skills in large language models and prompt engineering directly relevant to modern engineering practice.

B. Assignment Problem / Challenge

This assignment requires students to design, implement, and evaluate an AI-powered engineering assistant that applies core Generative AI concepts — prompt design and engineering techniques (zero-shot, few-shot, and chain-of-thought prompting), large language model (LLM) API integration, and structured evaluation of model outputs — to a unified engineering-support platform called **Prompt Craft**. Students must integrate multiple prompting strategies, build task-specific engineering modules, deploy the system as a lightweight web application, validate output quality and reliability with sample test cases, and address basic responsible-AI considerations such as transparency of prompts, hallucination handling, and reproducibility of results.

C. Problem Statement

Scenario: “PromptCraft” – A Prompt-Driven Engineering Assistant

Many students and practicing engineers find it difficult to consistently get high-quality, reliable outputs from large language models without structured prompting. Your task is to design and implement an interactive engineering-support web application called **PromptCraft** that helps engineers construct effective prompts, apply prompt engineering techniques, and use an LLM to accelerate common software-engineering tasks through a unified interface.

The system shall integrate three major functional modules:

1. Prompt Engineering Workbench

Implement zero-shot, few-shot, and chain-of-thought prompting templates for common engineering tasks.

- Users should be able to enter a task description, select a prompting strategy, and optionally supply few-shot examples or context.
- The system must display the fully constructed prompt sent to the LLM together with the generated response, enabling students to see exactly how prompt structure affects output quality.

2. LLM-Powered Engineering Task Modules

Implement at least three engineering-assistant task modules, e.g., code generation & explanation, automated debugging assistance, and requirement-to-documentation generation.

- Users will enter their engineering artifact (a code snippet, an error/bug description, or a requirement statement) as input to the selected module.
- The system should show the underlying prompt template used for each module, the LLM output, and any post-processing applied, so that students can trace how the assistant arrived at its result.

3. Prompt Strategy Evaluation & Comparison

Implement comparison of at least three prompting strategies (e.g., zero-shot, few-shot, chain-of-thought) for the same task.

- Users will enter a representative engineering task and a set of reference/expected outputs for evaluation.
- The system should display response quality, response time / latency, and token usage for each strategy, allowing clear comparative analysis.

Design the application workflow so that user inputs (task descriptions, code snippets, bug reports, requirement statements) are processed through the corresponding prompt templates and LLM calls, and the results are presented as clear prompts, responses, and comparative metrics. Develop a clean, simple web interface using **Streamlit**. Provide 2–3 sample test cases for each module to demonstrate effectiveness. Students must use a real or simulated LLM API (e.g., OpenAI, Anthropic, or an open-source model) to generate responses.

Evaluate the application under suitable test cases and analyze the quality of LLM outputs, usability of the interface, and clarity of the prompt/response traces. The completed system should also briefly address responsible-AI considerations including transparency of prompts, hallucination and error handling, data privacy, and reproducibility of results.

D. Requirements and Constraints

- **Functional:** Provide interactive modules for a Prompt Engineering Workbench (with visible prompt construction and response display), LLM-Powered Engineering Task Modules (code generation, debugging assistance, and documentation generation), and Prompt Strategy Evaluation (quality, latency, and token-usage comparison). Accept user inputs as well as sample data. Produce comparative tables and prompt/response traces.

- **Interface:** Clean, simple, and user-friendly Streamlit web application with separate pages or clearly separated sections for each module.
- **Performance:** LLM calls must return results with acceptable latency for interactive classroom use and support clear comparative analysis across prompting strategies.
- **Technical constraint:** Use Python + Streamlit. No database or complex architecture required. Use any accessible LLM API or open-source model for prompt execution.
- **Reliability:** Prompt templates and task modules must behave consistently and reproducibly; intermediate prompts must be transparent; the system must handle malformed inputs and LLM API errors without crashing.
- **Resource constraint:** Project must be completable within the semester timeline using standard student computing resources.
- **Responsible Computing:** Ensure transparency of prompts and model limitations, reproducibility of results, mitigation of hallucinated or misleading outputs, and adherence to responsible-AI and data-privacy practices.
- **Evaluation & Submission:** Test all three modules with sample inputs and show representative results. Submit working Streamlit code, a short report containing sample prompts/outputs / screenshots, comparative analysis, and a GitHub repository link.

E.Student Work

1. Problem Understanding and Formulation

Understanding of the Problem

The given assignment focuses on designing and implementing PromptCraft, a Prompt-Driven Engineering Assistant using Large Language Models (LLMs) and prompt engineering techniques. The main purpose of the system is to help students and engineers obtain better and more reliable outputs from AI models for common software-engineering tasks.

Many users directly enter simple or unclear questions into an LLM and may receive incomplete, incorrect, or inconsistent responses. PromptCraft addresses this problem by providing a structured environment where users can construct effective prompts and select suitable prompting strategies.

The system mainly supports three prompting techniques:

- Zero-Shot Prompting
- Few-Shot Prompting
- Chain-of-Thought Prompting

PromptCraft also provides engineering-support modules for:

1. Code Generation and Explanation
2. Automated Debugging Assistance
3. Requirement-to-Documentation Generation

Another important feature is the comparison of different prompting strategies for the same task. The system evaluates generated responses using factors such as response quality, response time, and token usage.

Expected Outcomes

- Improved quality of prompts.
- Better LLM-generated engineering solutions.
- Transparent display of constructed prompts.
- Comparison of different prompting strategies.
- Assistance with coding, debugging, and documentation.
- Improved understanding of how prompt design affects AI output.

Assumptions

- The user has access to the PromptCraft web application.
- A real or simulated LLM API is available.
- Users provide valid engineering-related inputs.
- The application is developed using Python and Streamlit.
- Internet access may be required when using an online LLM API.

Constraints

- Python and Streamlit are used.
- The interface should be simple and user-friendly.
- At least three prompting strategies are supported.
- At least three engineering task modules are implemented.
- Prompts and responses must be visible to users.
- The system should handle invalid inputs and API errors.
- Responsible AI and data privacy should be considered.

2. Application of Course Knowledge

Part I – Prompt Engineering Techniques

2.1 Zero-Shot Prompting

Zero-shot prompting means giving an instruction directly to the LLM without providing any examples.

Sample Prompt

Task: Explain the following Python program.

```
num = 10
if num % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Zero-Shot Prompt: Explain the following Python program in simple terms. Describe the purpose of each important statement and provide the final output.

Expected Output

The program checks whether the number stored in the variable num is even or odd. Since $10 \% 2$ is equal to 0, the program prints: Even.

Observation

Zero-shot prompting is simple and fast because no examples are required. However, the quality of the response depends strongly on how clearly the instruction is written.

2.2 Few-Shot Prompting

Few-shot prompting provides one or more examples to the LLM before asking it to solve a new task.

Example

Example Input: Code: `print("Hello")` Explanation: This program displays the word Hello.

Actual Task: `x = 5, y = 10, print(x + y)`

Few-Shot Prompt: Example: Code: `print("Hello")` Explanation: This program displays the word Hello. Now explain the following Python code in the same format: `x = 5, y = 10, print(x + y)`.

Expected Output

The program stores the values 5 and 10 in two variables. It adds both values and displays the result.
Output: 15.

Observation

Few-shot prompting improves consistency because the LLM understands the expected response format from the provided examples.

2.3 Chain-of-Thought Prompting

Chain-of-thought prompting is useful for tasks requiring structured reasoning and analysis.

Sample Task

A Python program should calculate the average of three numbers, but the output is incorrect.

Prompt: Analyze the problem carefully. Identify the possible cause of the incorrect output, determine the required calculation, and then provide the corrected solution with a clear explanation.

Expected Result

a = 10

b = 20

c = 30

average = (a + b + c) / 3

print("Average =", average)

Observation

This approach is useful for complex engineering problems because it encourages structured analysis before producing the final answer.

3. LLM-Powered Engineering Task Modules

PromptCraft contains three major engineering-support modules.

Module 1: Code Generation and Explanation

This module helps users generate code from a requirement or understand existing source code.

Sample Test Case 1

Input: Write a Python program to check whether a number is positive, negative, or zero.

Generated Code:

```
num = int(input("Enter a number: "))
```

```
if num > 0:
```

```
    print("Positive")
```

```
elif num < 0:
```

```
    print("Negative")
```

```
else:
```

```
    print("Zero")
```

Sample Test Case 2

```
for i in range(5):
```

```
    print(i)
```

Generated Explanation: The for loop runs five times. The variable i takes values from 0 to 4, and each value is printed.

Sample Test Case 3

Input: Write a Python program to find the largest of two numbers.

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
if a > b:
    print("Largest:", a)
```

else:

```
    print("Largest:", b)
```

Result: The module successfully converts user requirements into simple Python programs and provides explanations for existing code.

Module 2: Automated Debugging Assistance

This module accepts incorrect code or an error description and provides possible corrections.

Sample Test Case 1

```
if x > 5
```

```
    print(x)
```

Identified Error: The if statement is missing a colon (:).

```
if x > 5:
```

```
    print(x)
```

Sample Test Case 2

```
print("Hello"
```

Identified Error: The closing parenthesis is missing.

```
print("Hello")
```

Sample Test Case 3

```
num = "10"
```

```
result = num + 5
```

```
print(result)
```

Identified Error: A string cannot be directly added to an integer.

```
num = int("10")
```

```
result = num + 5
```

```
print(result)
```

Result: The debugging module identifies common syntax and type errors and provides corrected code with explanations.

Module 3: Requirement-to-Documentation Generation

This module converts a simple software requirement into structured documentation.

Sample Test Case 1

Input Requirement: Develop a system for managing student attendance.

Generated Documentation: System Name: Student Attendance Management System. Main Features: Student registration, attendance marking, attendance report generation, and record management.

Functional Requirements:

4. The system should allow students to be registered.
5. Faculty should be able to mark attendance.
6. Attendance records should be stored.
7. Reports should be generated when required.

Sample Test Case 2

Input Requirement: Develop a simple online library management system.

Main Features: Book management, user registration, book issue and return, and search functionality.

Sample Test Case 3

Input Requirement: Create a task management application.

Features: Create tasks, update tasks, delete tasks, and track task status.

Result: The module converts unstructured requirements into organized software documentation.

4. Prompt Strategy Evaluation and Comparison

The PromptCraft system compares the three prompting strategies by executing the same task with different prompt formats.

Representative Task

Write a Python program to check whether a number is prime.

Prompting Strategy	Response Quality	Response Time	Token Usage	Observation
Zero-Shot	Good	Low	Low	Simple and fast
Few-Shot	Very Good	Medium	Medium	More consistent output
Chain-of-Thought	High	Medium	High	Better for complex analysis

Evaluation Criteria

1. Correctness – Whether the generated response correctly solves the given task.
2. Relevance – Whether the response directly addresses the user's request.
3. Clarity – Whether the response is easy to understand.
4. Response Time – The time required for the LLM to generate a response.
5. Token Usage – The approximate number of tokens used in the prompt and response.

Conclusion

For simple tasks, zero-shot prompting is sufficient. Few-shot prompting is useful when a specific output format is required. Chain-of-thought prompting is more suitable for complex debugging and analytical tasks.

5. Solution / Design / Methodology

System Architecture

User



Streamlit Web Interface



Task Module Selection



Prompt Strategy Selection



Prompt Construction



LLM API / Model



Generated Response



Evaluation and Display

Step 1: User Input

The user enters an engineering task such as a programming requirement, source code, an error message, or a software requirement.

Step 2: Strategy Selection

The user selects one of the following strategies: Zero-Shot, Few-Shot, or Chain-of-Thought.

Step 3: Prompt Construction

The system automatically constructs a suitable prompt template based on the selected strategy.

Step 4: LLM Processing

The constructed prompt is sent to the selected LLM API or model.

Step 5: Response Generation

The LLM generates the required response.

Step 6: Result Display

The application displays the user input, constructed prompt, generated response, and evaluation information. This transparent workflow helps users understand the relationship between prompt design and output quality.

6. Use of Modern Tools

1. Python

Python is used as the main programming language because it is simple and supports AI-related libraries.

2. Streamlit

Streamlit is used to create the web interface. It allows users to enter tasks, select prompting strategies, and view generated responses.

3. Large Language Model API

An LLM API or open-source model is used to generate responses for engineering tasks.

4. Prompt Engineering

Prompt templates are created for Zero-Shot Prompting, Few-Shot Prompting, and Chain-of-Thought Prompting.

5. Performance Measurement

Basic measurements such as response time and token usage are used to compare different strategies.

The interface can include a text input area, strategy selection box, module selection, generate button, prompt display area, response display area, and comparison table.

7. Results and Validation

The PromptCraft application was tested using sample inputs for each module.

Test Results

Module	Test Status	Result
Code Generation	Successful	Generated relevant Python code
Code Explanation	Successful	Produced understandable explanations
Debugging Assistance	Successful	Identified common errors

Documentation Generation	Successful	Created structured requirements
Prompt Comparison	Successful	Displayed differences between strategies

Validation

Prompt Transparency: The complete constructed prompt was displayed to the user.

Response Generation: The LLM generated responses based on the selected task and prompting strategy.

Error Handling: The application can provide appropriate messages when input is empty, invalid data is entered, or the LLM API is unavailable.

Comparative Analysis: Different prompting strategies were tested with the same engineering task to observe differences in output quality and performance.

Overall Result

The system successfully demonstrated the effect of prompt engineering on LLM-generated engineering assistance.

8. Analysis and Engineering Decision

Zero-Shot Prompting

Advantages: Simple to use, fast response, low prompt size, and suitable for basic tasks.

Limitations: Output may be inconsistent and depends heavily on instruction clarity.

Few-Shot Prompting

Advantages: Improves response consistency, provides examples to guide the LLM, and is useful for structured output.

Limitations: Requires additional examples and uses more tokens than zero-shot prompting.

Chain-of-Thought Prompting

Advantages: Suitable for complex problems, supports structured analysis, and is useful for debugging and reasoning.

Limitations: May require more processing and can increase response length and token usage.

Final Engineering Decision

PromptCraft should provide all three strategies rather than selecting only one.

Task Type	Recommended Strategy
Simple Code Generation	Zero-Shot
Structured Output	Few-Shot
Complex Debugging	Chain-of-Thought

This flexible design allows users to select the most suitable strategy based on the engineering task.

9. Broader Considerations and Student Reflection

Reliability and Safety

The system improves reliability by displaying the exact prompt used for generation. However, LLM-generated outputs should always be reviewed before being used in real software projects.

Data Privacy

Users should avoid entering sensitive information such as passwords, API keys, personal information, and confidential project data. The application should encourage responsible handling of user data.

Responsible AI

PromptCraft should inform users that AI-generated responses may contain errors or hallucinated information. Generated code and documentation should be verified before implementation.

Usability and Accessibility

The Streamlit interface is designed to be simple and easy to use. Users can select modules and prompting strategies without requiring advanced technical knowledge.

Student Reflection

Through this assignment, I learned how Generative AI and prompt engineering techniques can be applied to real software-engineering tasks.

I understood that the quality of an LLM response is strongly influenced by the structure and clarity of the prompt. I also learned the differences between zero-shot, few-shot, and chain-of-thought prompting.

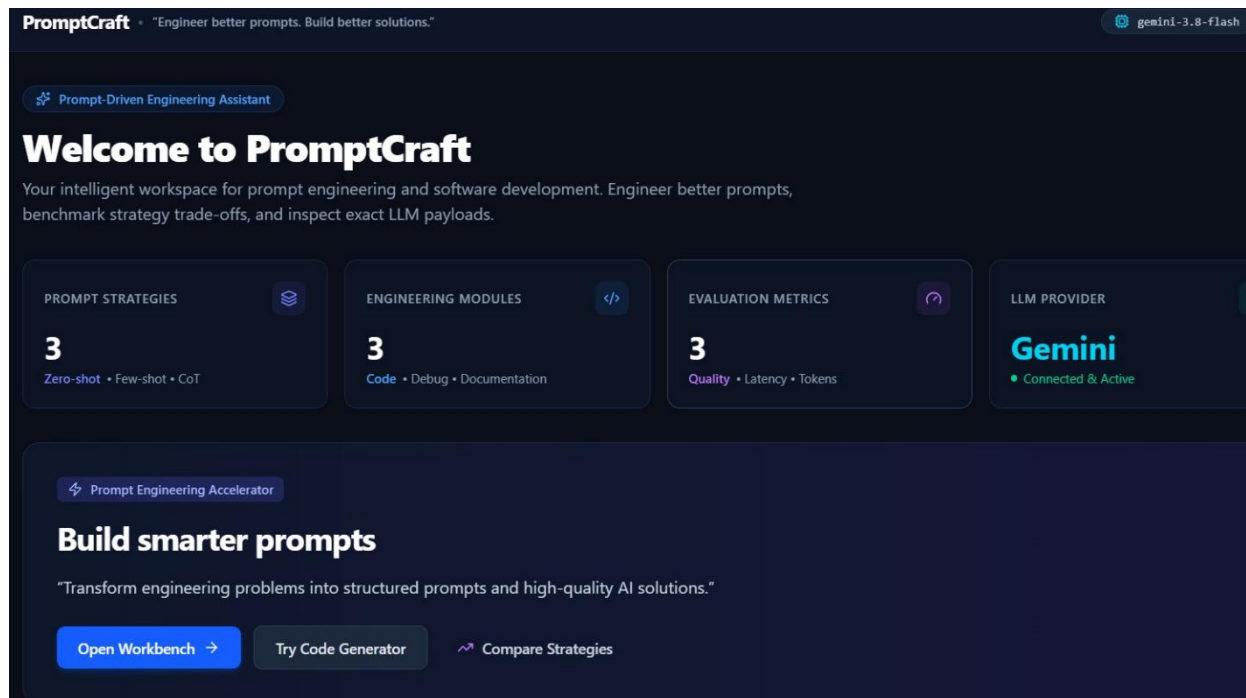
The assignment helped me understand how AI can assist with code generation, code explanation, debugging, and software documentation.

If additional time and resources were available, I would improve PromptCraft by adding support for multiple LLM models, automatic code testing, advanced evaluation metrics, and more engineering-support modules.

10. References

1. OpenAI, Large Language Model Documentation and API Documentation.
2. Streamlit, Streamlit Documentation.
3. Python Software Foundation, Python Documentation.
4. Brown, T. B. et al., “Language Models are Few-Shot Learners,” 2020.
5. Wei, J. et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” 2022.
6. Course materials for CSA08 – Generative AI and Prompt Engineering.

Implementation:



F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course / domain knowledge (Prompt Engineering, LLM Integration, Evaluation)	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques (Python, LLM APIs, Prompt Engineering Frameworks)	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO1/CO2	PO1, PO2	L3	10
Application – Prompt Engineering Techniques	CO1/CO2	PO1, PO2, PO3	L3/L4	8
Application – LLM Task Module Integration	CO1	PO1, PO2, PO3	L3/L4	6
Application – Evaluation & Comparison of Prompts	CO2	PO1, PO2, PO3	L3/L4	6
Solution / Design – Integrated PromptCraft	CO1–CO2	PO2, PO3, PO5	L4/L5/L6	20

Modern tool usage (Python, LLM APIs, Prompt Engineering)	CO1–CO2	PO5	L3	10
Validation of results	CO1–CO2	PO4	L4/L5	15
Analysis & justification / trade-offs	CO1–CO2	PO2, PO4	L4/L5	15
Broader considerations & documentation / reflection	CO1–CO2	PO6, PO12	L3/L4	10

Note: PO numbers indicated are illustrative; faculty shall verify against the current CSA08 CO–PO matrix and map only the POs genuinely assessed.

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60 – 69%
Level 1	50 – 59%
Level 0	$< 50\%$

Target CO Attainment: _____ Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering:
